Tingjian (Terence) Yu

Waterloo, ON | 548-398-2250 | terenceyyu@gmail.com | LinkedIn | Website

Profile

Seasoned software engineer with over 13 years of working experience. Expert in designing and developing scalable cloud-native applications based on event-driven microservices serving millions of clients. Adept at maintaining a balance between performance and consistency, delivering clean, testable code.

Work Experience

Senior Software Engineer **Dec 2017 – Mar 2023**

Ecovacs Robotics, China

- Led a group of 8 to build over a dozen backend applications, including NLP systems, IoT platforms, image processing, and data pipeline, using Go and Node.js, supporting over 10 million users and devices.
- Designed key architectural solutions with UML diagrams, focusing on event-driven microservices architecture, enhanced API design with Swagger, and relational modeling, ensuring scalability, security, and 99.99% high availability.
- Created service-oriented applications with Node.js, Go, Proto Buff, REST APIs, PostgreSQL, MongoDB, Kafka, and Redis, handling peak concurrent requests of around 30,000 per second.
- Incorporated asynchronous communication, caching, connection pooling, query optimization, data compression, and tools like pprof and Elastic APM to analyze and improve systems.
- Optimized performance through refactoring from monolith to microservices, rewriting JavaScript with Go and CGo, offloading server computation to edge devices, and reducing interface latency by up to 90%.
- Integrated third-party AI interfaces and built a labeling system to train product-related speech models using streaming gRPC, Redis, S3, PostgreSQL, Azure, and GCP, which increased the speech recognition rate by 7%.
- Streamlined software release processes by utilizing CI/CD tools like Git, Jenkins/GitHub Actions, SonarQube, and automated testing, shortening deployment time by 2/3, and reducing errors by 70%.
- Built and maintained system observability dashboards using Grafana and Prometheus.
- Collaborated closely with cross-functional teams to identify and prioritize work items in an Agile approach, delivering incremental software releases in 2-week sprints.
- Mentored junior engineers, improving team effectiveness through pair coding, code reviews, and technology sharing.

iOS Developer **May 2012 – Dec 2017**

Ecovacs Robotics, China

- Participated in the definition of mobile application prototypes and the design of UX and UI.
- Built the company's first robot-controlling mobile application from scratch.
- Developed iOS applications with OC and Swift, using Cordova to implement hybrid development.
- Encapsulated IoT communicating SDK, providing abstract interfaces to upper-level developers.

Embedded Software Developer **Mar 2010 – May 2012**

Qisda Corporation, China

- Built driver programs for Scanner's optical sensor and motor, manipulating registers to implement functions according to the chip's spec with C and Assembly language.
- Optimized firmware of Printers.

Skills

Languages: Go, JavaScript, Python, Java, C, TypeScript, Objective-C, Kotlin, Assembler, HTML.

Backend Development: Network Programming, Microservices Architecture, Distributed Systems, Design Patterns, REST, gRPC, GraphQL, OAuth, PostgreSQL, MySQL, MongoDB, Cassandra, ClickHouse, Redis, Kafka, RabbitMQ, S3.

Cloud and Tools: OpenAPI, Git, Agile, AWS, Azure, GCP, Docker, Kubernetes, CI/CD, Shell Scripting, Linux, Elastic Stack, Prometheus, Grafana, Terraform.

Education

Postgraduate Certificate of Web Development (Distinction) | *Conestoga College* 2023 – 2024

Bachelor of Electronic Information Engineering (Academic Scholarship) | *Chongqing University* 2005 – 2009

Work Projects as a Senior Software Engineer at Ecovacs

Machine Behavior Controlling based on Natural Language Processing **June 2021 – Apr 2022**

- Users needed to control their machines via voice interaction. My responsibilities included the design of the entire software system and the implementation of key codes.
  - Defined flowchart, database schema, Protobuf, and REST API.
  - Implemented a scalable backend architecture using Golang and gRPC to transmit encoded audio streams to third-party NLP platforms like Google Dialog Flow and Azure Speech-to-Text.
  - Decoupled third-party platforms' dependencies by creating modular integrations that can be easily replaced. Leveraged Google Cloud, Kafka, Redis, PostgreSQL, and S3 for efficient audio data processing and system scalability.
- Reached millions of users, with tens of millions of daily interactions three months after launch.

Machine Clean Logs Generating **June 2020 – Dec 2020**

- Users experienced serious latency when viewing clean logs on their mobile applications due to CPU-intensive operations running on Node.js based on single-thread and interpreting-running. My duty was to refactor the old project.
  - Rewrote the picture generation program using Golang and OpenCV. Compiled it into the ARM architecture library, which the machine loads as a dynamic library to offload server computation.
  - Employed technologies such as Goroutine, CGo, MongoDB, Kafka, and S3 to enhance data management and streaming capabilities.
- Decreased the log generation time from 1 second to 100 milliseconds, significantly improving performance.

Robot Open Platform and Smart Speaker Integration **Mar 2019 – Sept 2019**

- Third-party platforms demanded access to our customers' machines through their own applications. My task was to design a general platform and complete most of the coding work.
  - Designed multiple microservices, such as permission control and instruction scheduling, based on the single responsibility principle.
  - Developed a secure website and standardized APIs enabling third-party developers to register applications, set permissions, and execute operations via JWT and REST APIs after administrator approval, using Node.js, VUE, AWS Lambda, MongoDB, Oauth2.0, etc.
  - Implemented smart home applications based on Alexa and Google Home specifications to interact between speakers and customers' machines.
- Handled millions of daily API calls and thousands of concurrent requests per second at peak times.